

Lab 7 programm :

```
#include <stdio.h>

#define MAX 50

int search(int key, int frames[], int frameCount)
{
    for (int i = 0; i < frameCount; i++)
    {
        if (frames[i] == key)
            return 1;
    }
    return 0;
}

void FIFO(int pages[], int n, int frameCount)
{
    int frames[MAX];
    int index = 0;
    int faults = 0;
    for (int i = 0; i < frameCount; i++)
        frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
    for (int i = 0; i < n; i++)
    {
        if (!search(pages[i], frames, frameCount))
```

```

    {
        frames[index] = pages[i];
        index = (index + 1) % frameCount;
        faults++;
    }
    printf("Page %d -> [", pages[i]);
    for (int j = 0; j < frameCount; j++)
    {
        if (frames[j] != -1)
            printf("%d ", frames[j]);
    }
    printf("]\n");
}
printf("Total Page Faults (FIFO): %d\n", faults);
}

void LRU(int pages[], int n, int frameCount)
{
    int frames[MAX], time[MAX];
    int faults = 0, counter = 0;

    for (int i = 0; i < frameCount; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

```

```
}  
printf("\n--- LRU Page Replacement ---\n");  
for (int i = 0; i < n; i++)  
{  
    int found = 0;  
    for (int j = 0; j < frameCount; j++)  
    {  
        if (frames[j] == pages[i])  
        {  
            counter++;  
            time[j] = counter;  
            found = 1;  
            break;  
        }  
    }  
    if (!found)  
    {  
        int pos = -1;  
        for (int j = 0; j < frameCount; j++)  
        {  
            if (frames[j] == -1)  
            {  
                pos = j;  
                break;  
            }  
        }  
    }  
}
```

```

    }
}
if (pos == -1)
{
    int min = time[0];
    pos = 0;
    for (int j = 1; j < frameCount; j++)
    {
        if (time[j] < min)
        {
            min = time[j];
            pos = j;
        }
    }
}
frames[pos] = pages[i];
counter++;
time[pos] = counter;
faults++;
}
printf("Page %d -> [", pages[i]);
for (int j = 0; j < frameCount; j++)
{
    if (frames[j] != -1)

```

```

        printf("%d ", frames[j]);
    }
    printf("\n");
}
printf("Total Page Faults (LRU): %d\n", faults);
}

void Optimal(int pages[], int n, int frameCount)
{
    int frames[MAX];
    int faults = 0;

    for (int i = 0; i < frameCount; i++)
        frames[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");

    for (int i = 0; i < n; i++)
    {
        if (!search(pages[i], frames, frameCount))
        {
            int pos = -1;
            for (int j = 0; j < frameCount; j++)
            {
                if (frames[j] == -1)

```

```

    {
        pos = j;
        break;
    }
}
if (pos == -1)
{
    int farthest = -1;
    for (int j = 0; j < frameCount; j++)
    {
        int k;
        for (k = i + 1; k < n; k++)
        {
            if (frames[j] == pages[k])
                break;
        }
        if (k == n)
        {
            pos = j;
            break;
        }
        if (k > farthest)
        {
            farthest = k;
        }
    }
}

```

```

        pos = j;
    }
}
}
frames[pos] = pages[i];
faults++;
}
printf("Page %d -> [", pages[i]);
for (int j = 0; j < frameCount; j++)
{
    if (frames[j] != -1)
        printf("%d ", frames[j]);
}
printf("]\n");
}
printf("Total Page Faults (Optimal): %d\n", faults);
}

int main()
{
    int n, frameCount;
    int pages[MAX];
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");

```

```
for (int i = 0; i < n; i++)  
{  
    scanf("%d", &pages[i]);  
}  
printf("Enter number of frames: ");  
scanf("%d", &frameCount);  
FIFO(pages, n, frameCount);  
LRU(pages, n, frameCount);  
Optimal(pages, n, frameCount);  
return 0;  
}
```



```
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3
```

```
--- FIFO Page Replacement ---
```

```
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 3 1 ]
Page 0 -> [2 3 0 ]
Page 4 -> [4 3 0 ]
Page 2 -> [4 2 0 ]
Page 3 -> [4 2 3 ]
Page 0 -> [0 2 3 ]
Page 3 -> [0 2 3 ]
Total Page Faults (FIFO): 10
```

```
--- LRU Page Replacement ---
```

```
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 4 -> [4 0 3 ]
Page 2 -> [4 0 2 ]
Page 3 -> [4 3 2 ]
Page 0 -> [0 3 2 ]
Page 3 -> [0 3 2 ]
Total Page Faults (LRU): 9
```

```
--- Optimal Page Replacement ---
```

```
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 4 -> [2 4 3 ]
Page 2 -> [2 4 3 ]
Page 3 -> [2 4 3 ]
Page 0 -> [0 4 3 ]
Page 3 -> [0 4 3 ]
Total Page Faults (Optimal): 7
```

```
Process returned 0 (0x0)   execution time : 34.195 s
```



Search

